Source Code:

```
void setup(){
  PinMode(LED.BUILTIN,OUTPUT);
 }
void loop(){
  digital Write(LED_BUILTIN,HIGH);
  delay(1000);
  digital Write(LED_BUILTIN, LOW);
  delay(1000);
 }
```

Output: Turns the LED ON (High is voltage level
waits for a Second
Turns the LED OFF by making the Voltage
Low wait for a second.

Serial Monitor	9600	— □ X
LED is ON		
LED is OFF		
LED is ON		
LED is OFF		

Result: Hence, we Performed the Program to Perform blinking of LED ON & OFF using Arduino UNo.

Power: Requires 5V DC

PINS: Vcc, Trigger, Echo, Ground.

Applications:

1) Robotics: Obstacle avoidance, navigation,

2) Automation: Object detection, height detection

3) Home: Burglar alarms, Smart Parking ass

4) Education: Popular for hobbyist Projects

SOURCE PROGRAM:

```
#define ECHOPIN 8
#define TRIGPIN 7
int a,b;
void setup(){
  Serial.begin(9600);
  PinMode(ECHOPIN, INPUT);
  PinMode(TRIGPIN, OUTPUT);
}
void loop()
{
digital Write (TRIGPIN, LOW);
delay MicroseCond s (2000);
digital Write (TRIGPIN, HIGH);
delay MicroseCond s (1000);
digital Write (TRIGPIN, LOW);
Float a = PulseIn(ECHOPIN, HIGH);
b= a * 0.0344/2;
Serial.Print(b);
Serial.Println("cm");
delay(1000);
}
```

Program:

```
int IRPin =8;
int led=13;
int value;
void setup(){
PinMode(IRPin, INPUT);
Serial.begin (9600);
PinMode (led, OUTPUT);
}
void loop(){
value=digitalRead (IRPin);
Serial.Println (value);
if (digitalRead (IRPin)==0);
Serial.Println ("Object detected");
digitalWrite(led, HIGH);
}
else {
digitalWrite( led, LOW);
Serial.Println("Object not detected");
}
}
```

O/p:

```
Serial Monitor                    9600
0
object not detected
0
object not detected
1
object detected
0
object not detected
```

Result: Blinking of an LED /ON or OFF of an LED with respect to IR sensor using Arduino UNO.

2/12/25

3. The Arduino Bluetooth module at the other end receives the data and sends it to the Arduino through the TX pin of the Bluetooth module (connected to RX pin of Arduino).
4. The code uploaded to the Arduino checks the received data and compares it. If the received data is 1, the LED turns ON.
5. The LED turns OFF when the received data is 0. You can open the serial monitor and watch the received data while connecting.

SOURCE CODE Main Program:

```
#include <SoftwareSerial.h>
SoftwareSerial Bluetooth(9,8);
int LED=13;
int Data;
void setup(){
 Bluetooth.begin(9600);
 Serial.begin(9600);
 Serial.println("Waiting for Command...");
Bluetooth.println("Send 1 to turn on LED
Send 0 to turn off");
PinMode(LED, OUTPUT);
}
void loop(){
 if (Bluetooth.available()){
    Data=Bluetooth.read();
    if (Data=='1'){
     digitalWrite(LED,1)
     Serial.Println("LED on!");
     Bluetooth.println("LED On!");
    }
```



```
    else if (Data=='0'){
      digitalWrite(LED,0);
      Serial.println("LED off!");
      Bluetooth.Println("LED On Or off!");
    }
  else {;}
 }
 delay(100);
}
```

Basic Program:

```
#include <SoftwareSerial.h>
SoftwareSerial BTSerial(9,8);
void setup(){
      Serial.begin(9600);
BTSerial.begin(9600);
}
void loop(){
    Serial.Println("ece");
    BTSerial.print("c");
    BTSerial.Print("iii");
    BTSerial.print("W@L com@ To IoT Lab\n");
    BTSerial.Print(" ;");
    delay(3000);
}
```

OUTPUT

Serial Monitor

ece
ece
ece
ece

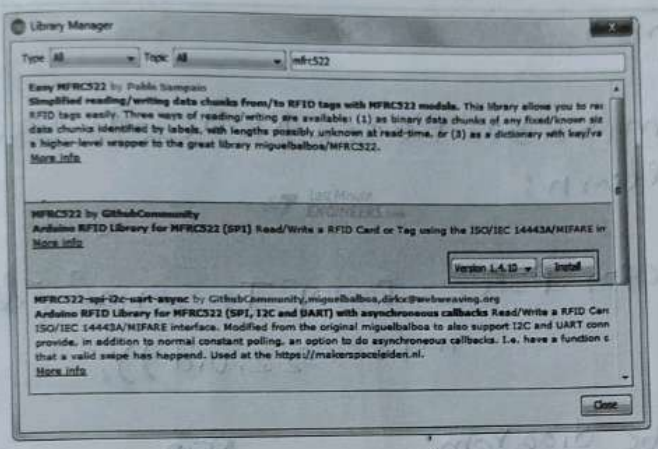
android phone

welcome to iot lab
welcome to iot lab
welcome to iot lab

output for Bluetooth & LED to Arduino

Enter 1 LED on
Enter 0 LED off

1 0 10



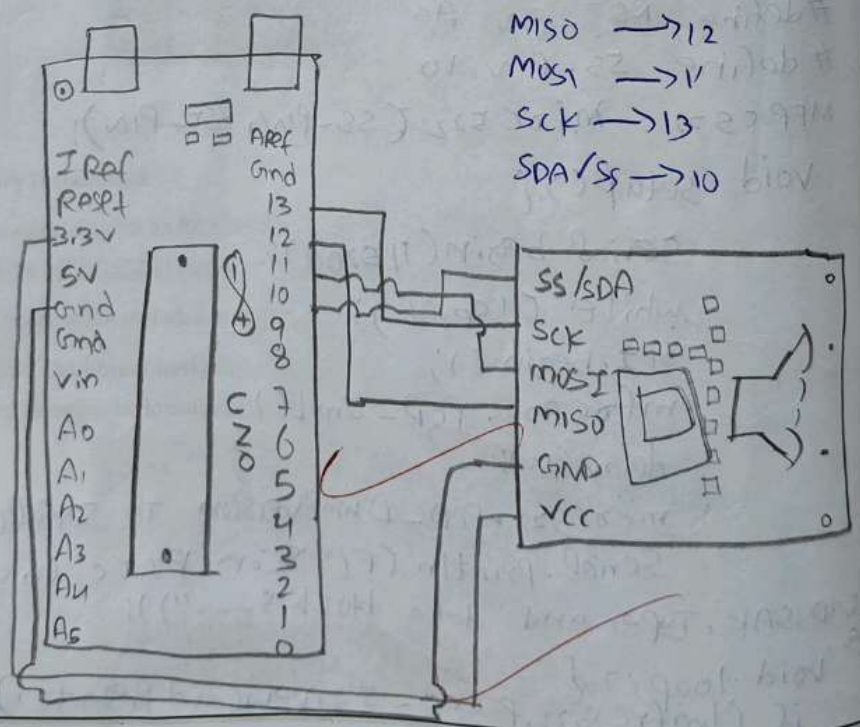
SOURCE CODE

```
#include <SPI.h>
#include <MFRC522.h>
#define RST_PIN A5
#define SS_PIN 10
MFRC522 mfrc522 (SS_PIN, RST_PIN);
void setup(){
    Serial.begin(115200);
    while (!Serial);
    SPI.begin();
    mfrc522.PCD_Init();
    delay(4);
    mfrc522.PCD_DumpVersion To Serial();
    Serial.println (F("Scan PICC to see
UID,SAK, Type, and data blocks---"));
void loop(){
  if (!mfrc522.PICC_IsNewCardPresent()) {
```

```
    return;
}
if(!mfrc522.PICC_ReadCardSerial()){
    return;
}
mfrc522.PICC_DumpToSerial(&(
        mfrc522.uid));
}
```

Schematic diagram:

RFID		Arduino
VCC	→	3.3V
Gnd	→	GND
RST	→	A5
MISO	→	12
MOSI	→	11
SCK	→	13
SDA/SS	→	10



OUTPUT

Card UID: EC 32 D3 C C

Card SAK: 08

Picc type: MIFARE 1KB

Sector	Block	0	1	2	3	4	5	6	7	8	9	10	11	12 ... 15
	63	00	00	00	00	00	FF	07	80	69	FF	FF	FF	FF
	62	00	00	00	00	00	00	00	00	00	00	00	00	00
	61	00	00	00	00	00	00	00	00	00	00	00	00	00
	60	00	00	00	00	00	00	00	00	00	00	00	00	00

Week - 5

Date: 19/1/26

AIM

Write a program to monitor temperature and humidity using Arduino.

HARDWARE REQUIREMENTS

1. Arduino UNO Board
2. DHT11/DHT22 Temperature and Humidity Sensor
3. Jumper Wires
4. Bread Board

PROCEDURE

1. Connect pin 1 (on the left) of the sensor to +5V.
2. Connect pin 2 of the sensor to whatever your DHTPIN.
3. Connect pin 4 (on the right) of the sensor to GROUND.

SOURCE CODE

```
# include "DHT.h"
# define DHT PIN 2
# defiro DHTTYPE DHT22
DHT dht (DHTPIN, DHTTYPE);
void setup(){
  Serial.begin(9600);
```



```
Serial.Println (F("DHTX test!"));
dht.begin();
}
void loop(){
  delay(2000);
  float h= dht.read Humidity();
  float t= dht.read Temperature();
  float f= dht.read Temperature (true);
if (is nan(h)|| is nan(t)|| isnan(f)){
  Serial.Println(F("tailed to read from DHT
  return;                    sensor!"));
}
float hit =dht.Compute Heat Index(f,h);
float hic=dht.ComputeHeat Index(t,h,false);
Serial.Print( F("humidity:"));
Serial.Print (h);
Serial.Print(F ("% Temperature:"));
Serial.Print (t);
Serial.Print (F (" °c"));
Serial.Print (f);
Serial.Print(F("°F Heat Index:"));
Serial.Print (hic);
Serial.Print (F ("°c"));
Serial.Print(hif);
Serial.Println (F("°F"));
}
```

OUTPUT

DHT TYPE DHT 11
Humidity: 95.00% Temperature: 30.06°C 87.08°F
Heat Index: 43.37°C 109.98°F

DHT TYPE DHT 22
Humidity: 73.90% Temperature: 30.30°C 86.54°F
Heat Index: 33.31°C 920.2°F

Result: I have performed a Program to Monitor Temperature and humidity using Arduino.

VIVA QUESTION

1. How much vol
Ans. The
on a V

2. What is DHT
Ans. The
that ma
output

3. What is the p
Ans. A Cap
to

4. How does A
Ans. By
sensor's
Rad

5. Name some
Ans. It is
Medical
safet

5. **SSID and password of your WIFI** connection should be given in source code
6. Compile and upload the program and verify the Sensor readings in serialmonitor
7. Go to cloud application(Thing speak) and Verify values in graph.

Hardware Requirements

1. Node MCU ESP8266 Board
2. IR sensor
3. Jumper wires
4. Bread board
5. WIFI Network

Secrets.h

```
#define SECRET_SSID "xxxxxxxxxxxxxxx"
#define SECRET_PASS "xxxxxxxxxx"
#define SECRET_CH_ID xxxxxxxxxxx
#define SECRET_WRITE_APIKEY "xxxxxxxxxxxxxxx"
```

SOURCE CODE

```
# include <ESP8266.WiFi.h>
# include <secrets.h>
# include "ThingSpeak.h"
Char SSID[ ]= SECRET_SSID;
Char Pass[ ]= SECRET_PASS;
int keyIndex=0;
WiFiClient client;
unsigned long myChannelNumber=SECRET_CH_ID;
Const char *myWriteAPIKey.
```

```
int number=0;
const int IRPIN=D3;
void setup(){
Serial.begin(115200);
while(!serial){
}
wifi.mode(WIFI_STA);
Thingspeak.begin(client);
}
void loop(){
if(wifi.status()!=WL_CONNECTED){
  Serial.Print("Attempting to Connect
      to SSID:");
Serial.Println(SECRET_SSID);
while(WiFi.status()!=WL_CONNECTED){
      WiFi.begin(SSid,Pass);
      Serial.Print(".");
      delay(5000);
    }
  Serial.Println("\n connected");
}
int x=Thingspeak.writeField(my channel
                Number,
  number, mywrite, API Key);
```



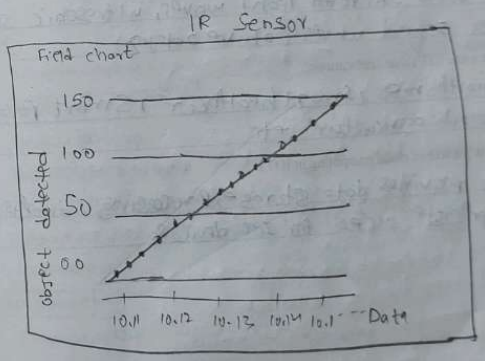
```
    if (x=200){
      Serial.Println("channel update
                successful");
    }
else {
  Serial.Println("Problem updating channel,
HTTP error code"+String(x))
    number++;
    if(number>999){
        number=0;
    }
    delay(20000);
}
```

Result: A program to monitor IR sensor data using IOT Cloud application is performed & verified.

OUTPUT

Serial Monitor - □ X

Channel update successfully.



SOURCE CODE

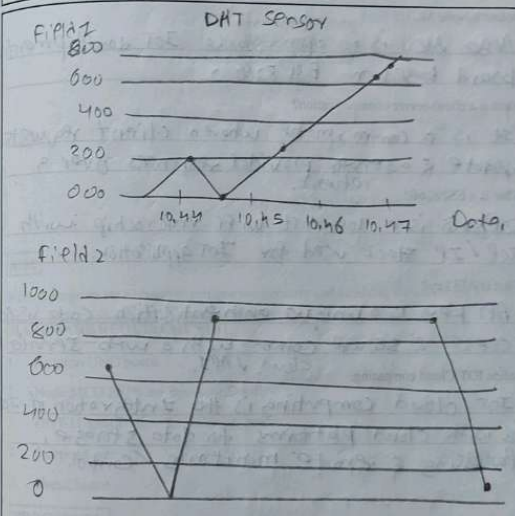
```
#include <ESP8266WiFi.h>
#include <secrets.h>
#include "ThingSpeak.h"
#include "DHT.h"
#define DHTTYPE DHT11
#define DHTPIN D3
DHT dht(DHTPIN, DHTTYPE);
char SSID[] = SECRET_SSID;
char Pass[] = SECRET_PASS;
int keyIndex=0;
WiFi client client;
unsigned long myChannelNumber =
                    SECRET_CH_ID;
const char *myWriteAPIKey = SECRET_WRITE_
                    APIKEY
int number1=0;
int number2 = random(0,100);
int number3 = random(0,100);
~~String~~ int number4=random(0,100);
String myStatus = " ";
void setup(){
    Serial.begin(115200);
    while(!Serial){ }
    WiFi.mode(WIFI_STA);
    ThingSpeak.begin(client);
}
```



```
void loop(){
  if (WiFi.status() != WL_CONNECTED){
    Serial.print("Attempting to Connected
        to SSID:");
  Serial.println(SECRET_SSID);
  while (WiFi.status() != WL_CONNECTED)
    WiFi.begin(SSID, Pass);
      Serial.print(" ");
      delay(5000);
  }}
  Serial.println("In Connected");
}
ThingSpeak.setField(1,number1);
ThingSpeak.setField(2,number2);
ThingSpeak.setField(3,number3);
ThingSpeak.setField(4,number4);
    if (number1 > number2){
  myStatus = String("Field 1 is greater than
                Field 2");
  }
else if(number1 < number2){
  myStatus = String("Field 1 is less than
  Field 2");
}
```

```
else
 my status = String("Field 1 equals to field 2");
int x = ThingSpeak.writeField5(my channel
    Number, my write APIkey);
 if (n=200);
    Serial.println("channel update successfully);
 } else {
  Serial.println(" Problem updating channel
HTTP Error code" + String(x));
 } number ++;
      if(number = 99) {
 }        number 1 = 0;
 number 2 = random(0,100);
 number 3 = random (0,100);
 number 4 = random (0,100);
  delay(20000);
 }
```


OUTPUT



Result: A program upload temperature & humidity data to store cloud using an Arduino is performed & verified.

SOURCE CODE

```
# include "Thingspeak.h"
# include <ESP8266WiFi.h>
Const char ssid[] = "SG";
Const char pass[] = "12345678";
int Status Code = 0;
WiFiClient client;
// --- Channel Details --- //
unsigned long Counter Channel Number = 2845211;
// Channel ID
Const char * my Counter Read APIKey = "";
Const int Field Number 1 = 1;
Const int Field Number 2 = 2;
void setup(){
  Serial.begin (115200);
  WiFi.mode (WiFi_STA)
  Thingspeak.begin(client)
}
void loop()
{
if (WiFi status() != WL_CONNECTED)
{
  Serial.Print("Connecting to");
  Serial.Print (ssid);
  Serial.Println ("...");
```

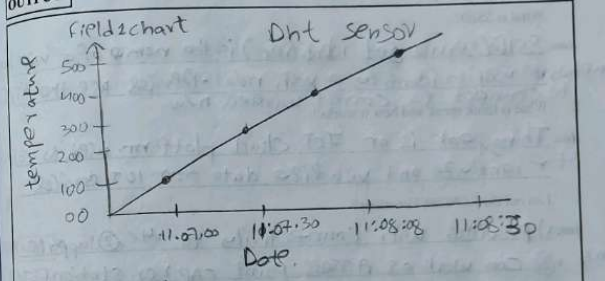


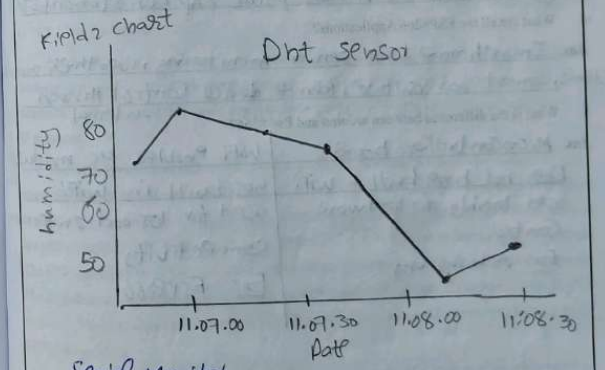
```
while(WiFi.status() != WL_CONNECTED)
{
  WiFi.begin(ssid, pass);
  delay(5000);
}
Serial.Println ("Connected to WiFi Successfully");
}
long temp = Thingspeak.readLongField (Counter
                    Channel Number,
  Field Number 1, my Counter Read APIKey);
Status Code = Thingspeak.getLastRead Status();
if (statusCode == 200) {
      Serial.Print (" Temperature");
      Serial.Println(Temp);
}else {
Serial.Println("unable to read channel / no
      internet connection"); }
      delay (100);
long humidity = Thingspeak.readLongField (Counter
  Channel Number, field Number 2, my Counter Read APIKey);
Status Code = Thingspeak.getLastRead status();
  if( Status Code == 200)
    { Serial.Print("Humidity:");
      Serial.Println( humidity);
    }
```

```
else
{
Serial.println("unable to read channel / No
                internal connection");
}
delay(100);
}
```

Result: Here Retrived the temperature and humidity from Cloud.

OUTPUT





Serial Monitor
temperature: 5
humidity: 51

The DHT11 Temperature and Humidity Sensor is a digital sensor with inbuilt capacitive humidity sensor and Thermistor. It relays a real-time temperature and humidity reading every 2 seconds. The sensor operates on 3.5 to 5.5 V supply and can read temperature between 0° C and 50° C and relative humidity between 20% and 95%. The DHT11 detects water vapors by measuring the electrical resistance between the two electrodes. The humidity sensing component is a moisture holding substrate with electrodes applied to the surface. When water vapors are absorbed by the substrate, ions are released by the substrate which increases the conductivity between the electrodes. The change in resistance between the two electrodes is proportional to the relative humidity. Higher relative humidity decreases the resistance between the electrodes, while lower relative humidity increases the resistance between the electrodes.

SOURCE CODE

```
#include "ESP8266WiFi.h"
#include "DHT.h"
#define DHTTYPE DHT11
Const char* ssid = "ASP";
Const char* Password = "12345678";
WiFiServer wifiServer(9000);
DHT dht(D4, DHT11);
void setup(){
  Serial.begin(115200);
  delay(1000);
  WiFi.begin(ssid, Password);
    while(WiFi.status() != WL_CONNECTED){
  delay(1000)
  Serial.Println("Connecting...");
}
```

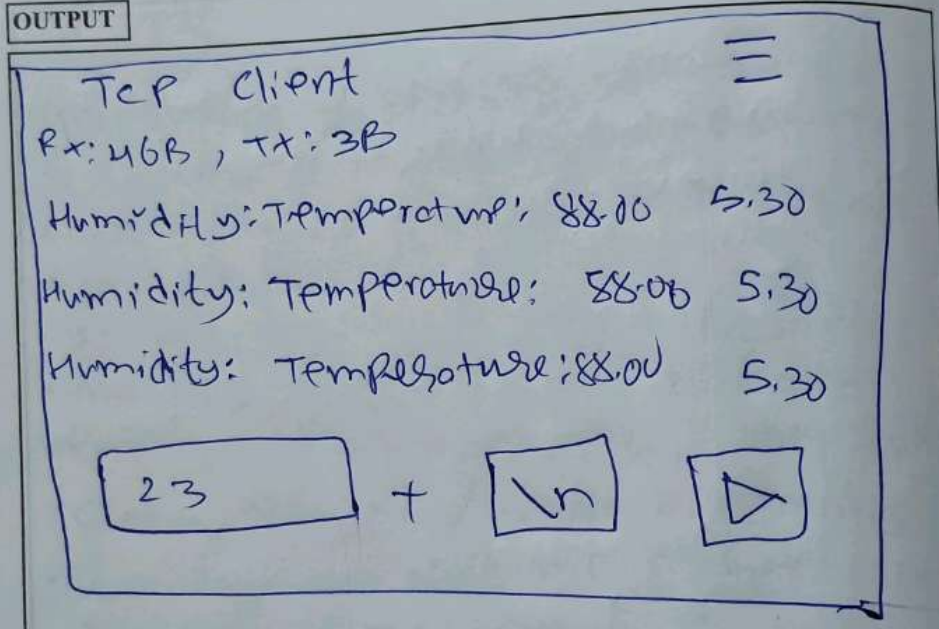


```
Serial.print("Connected to WiFi. IP:");
Serial.Println(WiFi.local IP());
WiFiServer.begin();
dht.begin();
}
void loop(){
  WiFiClient client = WiFiserver.available();
    if(Client){
      while(Client.connected()){
        while(client.available()>0){
  float t=dht.read Temperature();
  float h=dht.read Humidity();
  Client.print("humidity:");
  Client.Print("temperature:");
  Client.Println(h);
    Client.Println(t);
    Serial.Println(t);
    delay(2000);
    }}
    client.stop();
    Serial.Print ln("Client disconnected");
}}
```

Procedure:

- Open Arduino IDE software & write your Code & Connect to your WiFi & enter Password,
- Upload the Code into DHT11 sensor using node MCU module.
- It will generate a IP address. Copy that
- Now open your TCP Client APP in your mobile & connect same WiFi as your system,
- Click on settings & click on TCP settings
- Now enter your IP address generated & enter ~~Route~~ Port value as 9000.
- Now go back to home page & click on Connect.
- You will be connected, enter any value greater than zero.
- Now, you are able to visible the temperature & humidity values in your app & serial monitor.

OUTPUT



Result: Menu created TCP Server IP address & observed the temperature & humidity

VIVA QUESTIONS

1. Define TCP?

Ans. Transmi
oriented p
for

2. What is TCP Clien

Ans. A TCP
server
with

3. Explain TCP proto

Ans. TCP
destination
numb

4. How TCP protoc

Ans. TCP
acknowl

5. Write down the

Ans. TCP
C

SOFTWARE REQUIRED

- ThingSpeak server
- Arduino IDE

ESP8266 board needs to be loaded with the firmware code. In this tutorial, the firmware code is written using Arduino IDE. It is loaded to the ESP8266 board using the Arduino UNO. A generic ESP8266 board is used in this project. This board does not have any bootstrapping resistors, no voltage regulator, no reset circuit and no USB-serial adapter. The ESP8266 module operates on 3.3 V power supply with current greater than or equal to 250 mA. So, CP2102 USB to serial adapter is used to provide 3.3 V voltage with enough current to run ESP8266 reliably in every situation.

The ESP8266 Wi-Fi Module is a self contained SOC with integrated TCP/IP protocol stack that can access to a Wi-Fi network. The ESP8266 is capable of either hosting an application or off loading all Wi-Fi networking functions from another application processor. Each ESP8266 module comes pre-programmed with an AT command set firmware.

The Chip Enable and VCC pins of the module are connected to the 3.3 V DC while Ground pin is connected to the common ground. The chip enable pin is connected to VCC via a 10K pull up resistor. The RESET pin is connected to the ground via a tactile switch where the pin is supplied VCC through a 10K pull up resistor by default. The Tx and Rx pins of the module are connected to the RX and TX pins of the Arduino UNO. The GPIO-0 pin of the module is connected to ground via a tactile switch where the pin is supplied VCC through a 10K pull up resistor by default.

SOURCE CODE

```
#include <ESP8266WiFi.h>
#include <WiFiUdp.h>
#include <DHT.h>
Const char *SSID = "ASP";
Const char * Password = "1234567888";
Const char* UdpAddress = "192.168.0.7";
```

```
const int udpPort=8081
#define DHTPIN D4
  #define DHTTYPE DHT11
  DHT dht (DHTPIN, DHTTYPE);
WiFiUDP udp;
void setup(){
Serial.begin(115200);
  Serial.println();
  Serial.println("Connecting to WiFi...");
WiFi.begin(ssid, Password);
while (WiFi.status() != WL_CONNECTED){
  delay(2000);
  Serial.print(".");
}
Serial.println();
Serial.println("WiFi Connected");
  dht.begin();
}
void loop(){
    delay(2000);
    float temperature=dht.readTempera-
                        ture();
    float humidity=dht.readHumidity();
if (isnan(temperature)||isnan(humidity)){
Serial.println("Failed to read from DHT
    Sensor!");
  return;
}
```

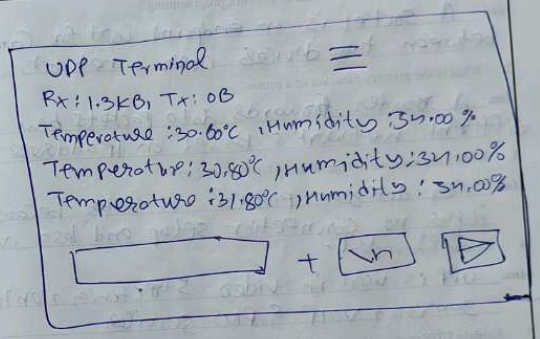


```
Serial.print("Temperature:");
  Serial.print(temperature);
  Serial.print(" Humidity:");
  Serial.print(humidity);
  Serial.println("%");
  Serial.println("Sending data over UDP...");
udp.beginPacket(udpAddress, udpPort);
  udp.print("Temperature:");
    udp.print(temperature);
    udp.print(" C, Humidity:");
    udp.print(humidity);
    udp.print ln("%");
    udp.endPacket();
    Serial.println("Data sent over UDP:");
}
```

Procedure:

- open Arduino IDE software & write your Code & connect to wifi & enter Password.
- Now open UDP terminal app in your mobile & click on Start.
- Before that dump the Code into DHT11 Sensor using NodeMCU module.
- Now enter Rx:IP address visible in app into Code.
- Connect DHT sensors to NodeMCU to (D4) Data Pin & dump
- Check your O/P in UDP terminal app will are able see the retrived data in your app.

OUTPUT



Result: Hence retrived the data using UDP IP address.